Experiment No.3

Title: Perform Bubble sort, Selection sort, Insertion, Quick and Merge sort.

Name of the student:-Gauravi Rajpal Shinde

Class:-SY1 Batch:-B1 PRN:B25ET1046 Div:-1

1.Bubblesort

```
#include <stdio.h>
void bubblesort(int arr[], int n) {
    int temp;

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    int arr[] = {98, 65, 78, 43, 32, 82};
    int n = 6;
    int i;

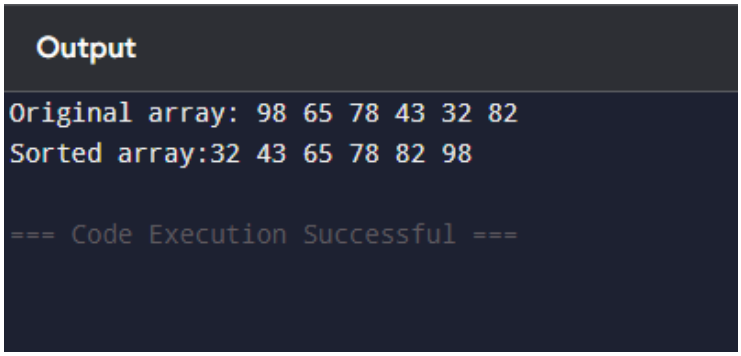
    printf("Original array: ");
    for (i=0;i<6;i++)
    printf("%d ",arr[i]);


    bubblesort(arr, n);

    printf("\nSorted array:");
    for (i=0;i<6;i++)
    printf("%d ",arr[i]);


    return 0;
}
```

Output:-



2.Selection sort

```
#include<stdio.h>
void selectionsort(int arr[], int n) {
    int minindex, temp;

    for (int i = 0; i < n - 1; i++) {
        minindex = i;

        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minindex]) {
                minindex = j;
            }
        }

        temp = arr[i];
        arr[i] = arr[minindex];
        arr[minindex] = temp;
    }
}
int main() {
    int arr[] = {98, 65, 78, 43, 32, 82};
    int n = 6;
    int i;

    printf("Original array: ");{
    for (i=0;i<6;i++)
    printf("%d ",arr[i]);}


    selectionsort(arr, n);

    printf("\nSorted array:");{
    for (i=0;i<6;i++)
    printf("%d ",arr[i]);}
```

```
 return 0;
}
```

Output:-

```
Output

Original array: 98 65 78 43 32 82
Sorted array:32 43 65 78 82 98

=== Code Execution Successful ===
```

3.Insertion sort

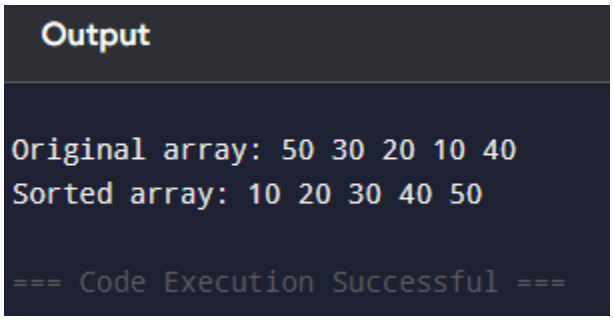
```
#include<stdio.h>
void insertionSort(int arr[], int n) {
    int i, j, key;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
int main()
{
    int arr[] = {50, 30, 20, 10, 40};
    int n = 5; int i;
    printf("\nOriginal array: ");
    for (i = 0; i < n; i++)
    printf("%d ", arr[i]);

    insertionSort(arr, n);

    printf("\nSorted array: ");
    for (i = 0; i < n; i++)
    printf("%d ", arr[i]);
    return 0;
```

```
    }
```

Output:-



4.Quicksort

```
#include <stdio.h>

void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;
    int j;

    for (j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }

    swap(&arr[i + 1], &arr[high]);

    return i + 1;
}
```

```
void quicksort(int arr[], int low, int high)
{
    if (low < high)
    {
        int p = partition(arr, low, high);

        quicksort(arr, low, p - 1);
        quicksort(arr, p + 1, high);
    }
}

int main()
{
    int arr[100];
    int n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);

    for (i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("Original array: ")
    for (i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }

    quicksort(arr, 0, n - 1);

    printf("\nSorted array: ");
    for (i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
```

```
    return 0;
}
```

Output:-

```
Output

Enter number of elements: 5
Enter 5 elements:
34,54,21,48,96
Original array: 34 0 0 0 0
Sorted array: 0 0 0 0 34

=== Code Execution Successful ===
```

5.MERGE SORT

```
#include <stdio.h>

void merge(int arr[], int l, int mid, int h);

void mergesort(int arr[], int l, int h)
{
    if (l < h)
    {
        int mid = (l + h) / 2;
        mergesort(arr, l, mid);
        mergesort(arr, mid + 1, h);
        merge(arr, l, mid, h);
    }
}

void merge(int arr[], int l, int mid, int h)
{
    int i = l;
    int j = mid + 1;
    int k = l;
    int temp[100];

    while (i <= mid && j <= h)
    {
        if (arr[i] < arr[j])
        {
            temp[k] = arr[i];
```

```
            i++;
        }
        else
        {
            temp[k] = arr[j];
            j++;
        }
        k++;
    }

    while (i <= mid)
    {
        temp[k] = arr[i];
        i++;
        k++;
    }

    while (j <= h)
    {
        temp[k] = arr[j];
        j++;
        k++;
    }

    for (i = l; i <= h; i++)
    {
        arr[i] = temp[i];
    }
}

int main()
{
    int arr[] = {50, 30, 20, 10, 40};
    int n = 5;
    int i;

    printf("\nOriginal array: ");

    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
```

```
    mergesort(arr, 0, n - 1);

    printf("\nSorted array: ");

    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
```

output:-

